

Windows Terminal & Python Dependency Management

Comprehensive Sandbox & Execution Integration Manual

Institute Handout — Systems & Software Architecture Group

Date: July 14, 2026

Lab Objectives

By the end of this intensive 2-hour lab session, students will be able to:

1. Natively navigate Windows file structures using Command Prompt (CMD) and PowerShell without graphical assistance.
2. Perform robust file mutations (create, copy, move, rename, delete) directly from shell consoles.
3. Isolate software runtime dependencies cleanly using Python's native `venv` engine.
4. Track, version-lock, and restore third-party application environments via `requirements.txt` manifests.
5. Execute modular utility programs, run in-memory one-liners, and intercept runtime processes interactively using advanced interpreter CLI flags (`-m`, `-c`, `-i`).

Safety Warning for Beginners

Isolation Guardrail & Irreversible Deletions:

- **No Recycle Bin:** Any files deleted using the terminal command lines bypass the Windows Recycle Bin completely. They are immediately and permanently purged from the hard drive. Double-check your path strings before running destruction actions.
- **Global Pollution Warning:** Always ensure your local environment prefix indicator (`(venv)` or `(.venv)`) is actively prepended to your command line prompts before invoking any `pip install` actions. This protects your system-wide Python installation from corruption.

1 Task 1: System Navigation & Spatial Awareness (25 Mins)

Before deploying virtual files, you must establish perfect spatial awareness of your position in the file directory tree. Think of the terminal as a dynamic map where you are represented by a path indicator.

1.1 Part A: Positioning and Inspecting in Command Prompt

1. Press the **Windows Key**, type `cmd`, and hit **Enter** to launch a native Command Prompt.
2. Locate your ****Current Working Directory**** to the left of the blinking block cursor (e.g., `C:\Users\YourName>`).

3. Execute the directory investigation command to inspect all internal directory listings:

```
dir
```

4. Analyze the terminal output. Point out how structural sub-directories are clearly annotated with a <DIR> indicator.

1.2 Part B: Positioning and Inspecting in PowerShell

1. Press the **Windows Key**, type `powershell`, and hit **Enter** to initialize the modern PowerShell shell wrapper.
2. Explicitly request your formal path using its dedicated cmdlet:

```
Get-Location
```

3. Now, list the contents of the folder using either of the unified modern POSIX aliases:

```
ls
```

1.3 Part C: The Art of Precision Navigation

Using `cd` (Change Directory), practice moving down and back up the path architecture.

1. **Move Forward:** Look closely at your directory listings, locate the **Desktop** folder, and descend inside it:

```
cd Desktop
```

2. **Step Backward:** Move precisely one folder level backward to return to your base user home folder:

```
cd ..
```

3. **Relative Deep Jump:** Skip manual transition steps and jump straight into deep structural folders:

```
cd Desktop\Documents
```

4. **Root Level Escape:** Jump all the way out of your current path and land straight at the partition baseline directory (`C:\`):

```
cd \
```

2 Task 2: Workspace Sandbox Initialization & File Operations (25 Mins)

Let's build a dedicated playground directory to manipulate workspace files without disturbing key folders.

2.1 Step-by-Step Directory Construction

1. Point your terminal back onto your physical Desktop. Replace YOUR_USERNAME with your actual Windows account profile name:

```
cd C:\Users\YOUR_USERNAME\Desktop
```

2. Create a new directory named TerminalSandbox:

```
mkdir TerminalSandbox
```

3. Move directly inside your newly structured testing space:

```
cd TerminalSandbox
```

4. **File Provisioning:** Create a fresh, empty text file labeled `notes.txt` using the correct shell utility:

Command Prompt (CMD):

```
type nul > notes.txt
```

PowerShell (PS):

```
New-Item notes.txt
```

2.2 Workspace File Mutations (Copying, Moving, & Renaming)

1. Create a dedicated nested subdirectory within the sandbox:

```
mkdir Archive
```

2. **Copying File Payloads:** Create an identical duplicate copy of `notes.txt` inside the Archive folder:

Command Prompt (CMD):

```
copy notes.txt Archive\
```

PowerShell (PS):

```
Copy-Item notes.txt Archive\
```

3. **Moving & Renaming Files:** Grab your original baseline `notes.txt` file, transfer it to a new location, and rename it to `todo.txt` simultaneously:

Command Prompt (CMD):

```
move notes.txt todo.txt
```

PowerShell (PS):

```
Move-Item notes.txt todo.txt
```

3 Task 3: Python Virtual Environments & Isolation Sandboxing (30 Mins)

With shell operations mastered, we will establish an isolated environment. This keeps our projects lightweight and prevents library version conflicts.

3.1 Sandbox Creation & Environment Activation

1. Inside your active shell path (Desktop\TerminalSandbox), create a secondary folder specifically to store our clean Python environments:

```
mkdir python_env_lab  
cd python_env_lab
```

2. Deploy Python's system virtual engine (venv) to compile local runtime binaries into a folder labeled .venv:

```
python -m venv .venv
```

3. **Activation Strategy:** Execute the correct script based on your active terminal structure: **Command Prompt (CMD):**

```
.venv\Scripts\activate.bat
```

PowerShell (PS):

```
.venv\Scripts\Activate.ps1
```

4. Verify that a (.venv) tag appears on the left side of your active command prompt pathway.

3.2 Dependency Modification Natively via pip

1. Run a list query on your fresh sandbox to confirm that no global user packages are polluting your space:

```
pip list
```

2. Install an older, targeted release of Django to practice pinning specific package versions:

```
pip install django==4.2.5
```

3. Rerun your listing tool to observe how the package installer automatically mapped and resolved nested dependencies:

```
pip list
```

4 Task 4: Generating and Restoring Ecosystem Blueprints (20 Mins)

When sharing codebases, you should never commit or transfer the heavy virtual environment directory. Instead, export a simple text recipe listing the exact packages you need.

1. **Freeze Environment:** Export your exact library version specs into a standard manifest file:

```
pip freeze > requirements.txt
```

2. Use your terminal directory viewer (`dir` or `ls`) to verify that the text-based `requirements.txt` file was created.
3. **Simulate System Failure:** To test your restore recipe, completely delete Django from your sandbox:

```
pip uninstall django -y
```

4. Verify that the package has been removed from the list:

```
pip list
```

5. **Ecosystem Restoration:** Rebuild your environment from your text recipe with a single command:

```
pip install -r requirements.txt
```

6. Confirm that Django and all of its dependencies have been successfully reinstalled:

```
pip list
```

5 Task 5: Harnessing Command Line Execution Flags (20 Mins)

The Python interpreter has several helpful runtime configurations you can pass during terminal calls.

5.1 Part A: Executing Modules as Utility Scripts (-m)

Using the `-m` flag, you can execute built-in libraries as standalone command-line programs.

1. Directly query your environment's packages using python's binary path:

```
python -m pip list
```

2. Start a lightweight local web server directly out of your current directory:

```
python -m http.server 8000
```

3. Open up your browser and visit <http://localhost:8000> to inspect your directory structure visually.
4. Go back to your terminal window and press **Ctrl + C** to stop the server.

5.2 Part B: Inline Code Execution (-c)

The `-c` flag allows you to run brief lines of Python code on the fly without making a temporary script file.

1. Run a simple print statement:

```
python -c "print('Direct terminal execution: OK!')"
```

2. Programmatically query your virtual environment's internal system path directory list:

```
python -c "import sys; print('\n'.join(sys.path))"
```

5.3 Part C: Interactive Inspection Mode (-i)

The `-i` flag keeps Python open in interactive mode after running a script, allowing you to inspect its variables and state live.

1. Create a quick calculations file named `math_test.py`:

Command Prompt (CMD):

```
echo x = 105 > math_test.py
```

PowerShell (PS):

```
"x = 105" | Out-File -Encoding ASCII math_test.py
```

2. Run the file standardly. Notice that the script runs and then immediately exits:

```
python math_test.py
```

3. Run the file again, but append the inspection flag to keep the process open:

```
python -i math_test.py
```

4. Your terminal command prompt changes to three chevrons (>>>). Inspect the variable `x` interactively:

```
print(x * 5)
```

5. Type the exit function to leave the interactive terminal:

```
exit()
```

6 Lab Exit & Comprehensive Cleanup Sequence

To conclude your session, follow these steps to reset your workspace. This leaves your system clean and organized.

1. **Deactivate Session:** Safely shut down and detach from your isolated python sandbox:

```
deactivate
```

2. Ensure that the `(.venv)` prefix is no longer visible on your command line.
3. Step back up out of your active environment sandbox:

```
cd ..
```

4. **Purge All Local Folders:** Permanently delete your files, requirements, scripts, and directories. *Remember, this bypasses the Recycle Bin.*

Command Prompt (CMD):

```
rmmdir /s /q python_env_lab  
cd ..  
rmmdir /s /q TerminalSandbox
```

PowerShell (PS):

```
Remove-Item -Recurse -Force python_env_lab  
cd ..  
Remove-Item -Recurse -Force TerminalSandbox
```

5. Clear your command history completely so that your terminal screen is blank before calling your instructor for final grading:

Command Prompt (CMD):

```
cls
```

PowerShell (PS):

```
Clear-Host
```